Project Proposal: Disaster Database (Final Implementation)

Ada Şevval Sari
Student ID: 220303023
Date: 19 January 2026

Abstract

This project implements a relational database called *DisasterDB* for storing disaster events and response operations in a consistent, queryable structure. Compared to the midterm design, the final schema was **fully implemented in Microsoft SQL Server (T-SQL)** and simplified for practical use: the separate *countries* table and the *early_warnings* weak entity were removed, and references were redesigned to use **natural keys** (e.g., *event_label*, *org_name*, *sector_name*) for readability and easier reporting. The implementation includes primary keys, unique constraints, foreign keys, check constraints, and multiple triggers that enforce data quality (e.g., valid event durations, normalized delivery methods, status validation, trimming/normalization, and preventing deletion of events with operational records).

Arel Üniversitesi, Müh. Mim. Fak. Bilgisayar Mühenendisliği. Türkoba Mah., Erguvan Sok., No:26, K 34537, Tepekent – Büyükçekmece, Istanbul

1 Introduction

Disaster information is multi-dimensional: a disaster must be described by its hazard type, location, time interval, and magnitude, and it must also be connected to impact summaries and operational actions such as aid delivery and organizational response. The goal of *DisasterDB* is to provide a structured model that supports analytics and reporting questions such as: *Which hazards occurred where and when? What impacts did they cause? Which organizations responded, with what actions and aid, and how quickly?*

In the final implementation, the schema was refined to support real-world insertion and querying in SQL Server. The database now uses stable descriptive identifiers (natural keys) to improve interpretability in joins and reports.

2 Project Plan and Objectives

The main objectives of this project are:

1. **Finalize the Data Model:** Refine the midterm design into an implementation-ready schema with clear entities and relationships.
2. **Implement in SQL Server (T-SQL):** Create all tables, keys, constraints, and enforcement logic in SQL.
3. **Ensure Data Integrity:** Add check constraints and triggers (duration logic, normalization rules, basic validations).
4. **Prepare Sample Data and Test Queries:** Insert realistic sample datasets and verify joins across events, impacts, sectors, organizations, aid, and actions.

Scope: A database that represents the disaster lifecycle from event description and location context to impacts and operational response, implemented as a single coherent relational system.

3 System Design and Scope

The final version of *DisasterDB* is implemented as a relational database consisting of **10 tables** organized into **five logical domains**. The design reflects the full lifecycle of a disaster, from location and hazard definition to impact assessment and operational response.

- **A) Geography** Geographic information (country, city, administrative region, and optional coordinates) is stored directly in *events* to simplify querying and reporting.
- **B) Hazards** *hazard_types* and *hazard_subtypes* classify disasters by general type and refined subtype. Subtypes reference their parent hazard type using name-based foreign keys.
- **C) Events (Core)** *events* represents individual disaster occurrences with temporal, spatial, and descriptive attributes. Each event is uniquely identified by an *event_label* to support readable reporting and value-based references.
- **D) Impacts** *event_impacts* stores summarized human and economic losses per event, while *sectors* and *event_sector_impact* model sector-specific disruptions using an N:M relationship.
- **E) Operations** *organizations* models responding entities with an optional recursive parent organization structure. *aid* and *org_event_actions* capture financial assistance and operational actions linked to events and organizations.

Entity Relationship Overview:

1. One hazard type may have multiple hazard subtypes.
2. One event may have one impact summary and multiple sector impacts.
3. One event may receive aid and actions from multiple organizations.
4. Organizations may form hierarchical structures through a parent organization relationship.

4 Results

A realistic sample dataset was inserted to test the final schema, including global and country-level disasters, organizations, impacts, sector impacts, aid deliveries, and operational actions.Integrity was verified through foreign keys, check constraints (e.g., nonnegative impacts, valid latitude/longitude, nonnegative magnitudes), and triggers (e.g., preventing invalid event durations, normalizing delivery methods, validating action statuses, and preventing deletion of events that already have operational records).

The following figure and table provide general statistics and a compact example summary format suitable for reporting.

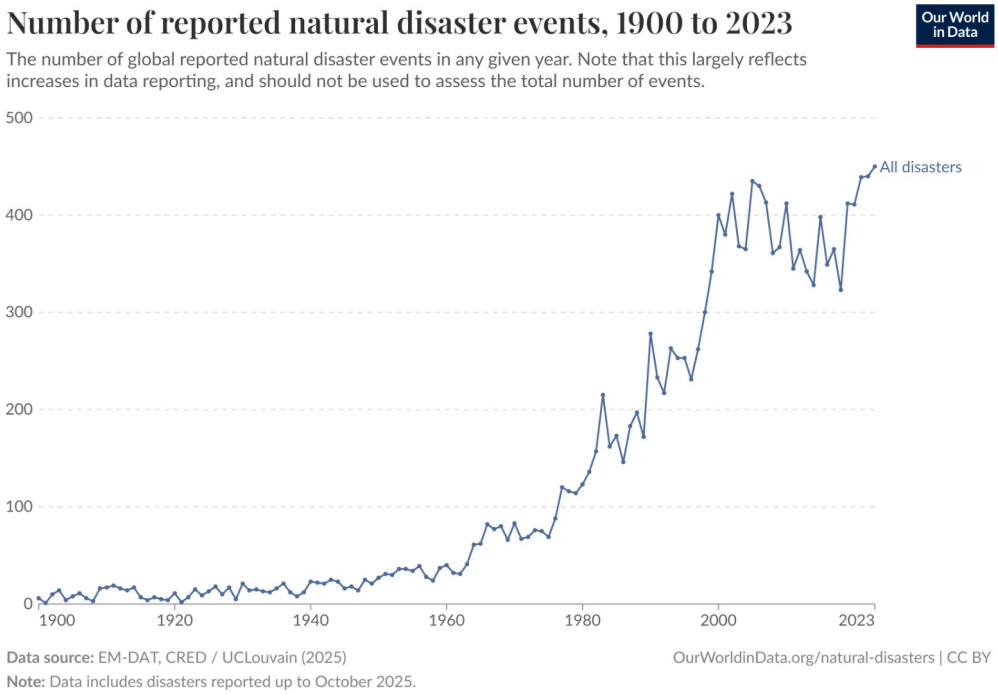

Figure 1: Number of reported natural disaster events worldwide, 1900–2023 (Our World in Data, 2025)

The final *DisasterDB* schema supports multiple reporting perspectives through SQL aggregation and joins. Event coverage is analyzed by hazard type, while impact summaries evaluate losses and displacement. Sector-level analysis joins events with affected sectors and ranks them by disruption. Operational reporting combines actions and aid records into timelines, demonstrating that the schema supports analytical queries.

5 Remarks and Recommendations

The final *DisasterDB* implementation is stable and practical for reporting because it includes integrity enforcement (constraints + triggers) and uses readable natural-key references for core reporting dimensions. Future extensions can add new capabilities without breaking the current structure.

Possible database extensions include:

- Re-introducing an **early_warnings** component as a weak entity if alert history needs to be tracked per event.
- Adding a **resources** table to record supplies/equipment (type, quantity, unit, assigned organization, event linkage).
- Adding **volunteers/staff** and **deployment** tables to track human resources and assignments.
- Creating an **attachments/media** table to store reports, documents, and links per event (with metadata and versioning).
- Adding a **recovery_projects** module to represent reconstruction activities, budgets, and milestones.

These additions would expand analytical capability while keeping the core event-impact-operations model consistent.

References

1. R. Elmasri, S. B. Navathe, *Fundamentals of Database Systems*, 7th Ed., Pearson, 2016.
2. Coppola, D. P., *Introduction to International Disaster Management*, 4th Ed., Butterworth-Heinemann, 2020.
3. Haddow, G. D., Bullock, J. A., *Introduction to Emergency Management*, 7th Ed., Butterworth-Heinemann, 2022.
4. Microsoft, *SQL Server Documentation (Constraints, Triggers, and Data Integrity)*, https://learn.microsoft.com/
5. DbDiagram, *DBML Reference Guide*, https://www.dbml.org/
6. United Nations Office for Disaster Risk Reduction (UNDRR), *How Information Systems Support Disaster Risk Reduction*, https://www.undrr.org/